

✓ Cost-Aware Credit Card Fraud Detection System

Objective

Develop an explainable and cost-aware fraud detection framework that identifies fraudulent transactions and prioritizes high-value cases using a LossScore ranking strategy to maximize fraud value recovery under limited review capacity.

Dataset:

- IEEE-CIS Fraud Detection Dataset
- 100,000 transactions

Tech Stack:

- Python
- Pandas
- NumPy
- XGBoost
- Scikit-Learn
- SHAP
- Matplotlib

Dataset Scope

The original IEEE-CIS Fraud Detection dataset contains approximately 590,000 transactions.

A 100,000-transaction subset was used to accelerate experimentation, feature analysis, threshold evaluation, and business-impact simulations while preserving the original fraud distribution.

Future work will evaluate the framework on the complete dataset.

✓ Business Problem

Financial institutions process millions of transactions daily.

Traditional fraud detection models focus on maximizing classification metrics such as accuracy or ROC-AUC.

However, not all fraud cases have the same financial impact.

Missing a \$5 fraud and missing a \$3000 fraud have very different business consequences.

This project investigates:

1. How accurately fraudulent transactions can be detected.
2. Which variables contribute most to fraud predictions.
3. How classification thresholds affect fraud detection performance.
4. Whether fraud review can be prioritized using financial risk instead of fraud probability alone.

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_auc_score,
    precision_score,
    recall_score
)

from xgboost import XGBClassifier
```

```
import shap

import warnings
warnings.filterwarnings("ignore")
```

```
from google.colab import drive
drive.mount('/content/drive')

df = pd.read_csv(
    "/content/drive/MyDrive/ieee-fraud-detection/merged_fraud_data.csv",
    nrows=100000
)

print(df.shape)
```

```
Mounted at /content/drive
(100000, 434)
```

Dataset Overview

This section examines:

- Dataset size
- Fraud distribution
- Missing values
- Data quality

```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

print("\nFraud Distribution")

print(
    df["isFraud"]
    .value_counts(normalize=True)
    * 100
)
```

```
Rows: 100000
Columns: 434
```

Fraud Distribution

```
isFraud
0    97.439
1     2.561
Name: proportion, dtype: float64
```

Missing Value Investigation

Many fraud datasets contain substantial missing information.

Instead of immediately imputing missing values, we first investigate whether missingness itself carries fraud-related information.

Hypothesis:

Transactions with missing values may have different fraud rates compared to transactions without missing values.

```
missing_pct = (
    df.isnull()
    .mean()
    .sort_values(ascending=False)
    * 100
)

missing_df = pd.DataFrame({
    "Missing_Percent": missing_pct
})

missing_df.head(15)
```

	Missing_Percent
id_24	98.500
id_25	98.388
id_21	98.380
id_07	98.377
id_26	98.377
id_08	98.377
id_23	98.376
id_27	98.376
id_22	98.376
D7	93.928
dist2	91.769
D13	90.479
D12	89.495
D14	88.847
D6	87.259

Next steps:

[Generate code with missing_df](#)[New interactive sheet](#)

```
plt.figure(figsize=(8,5))

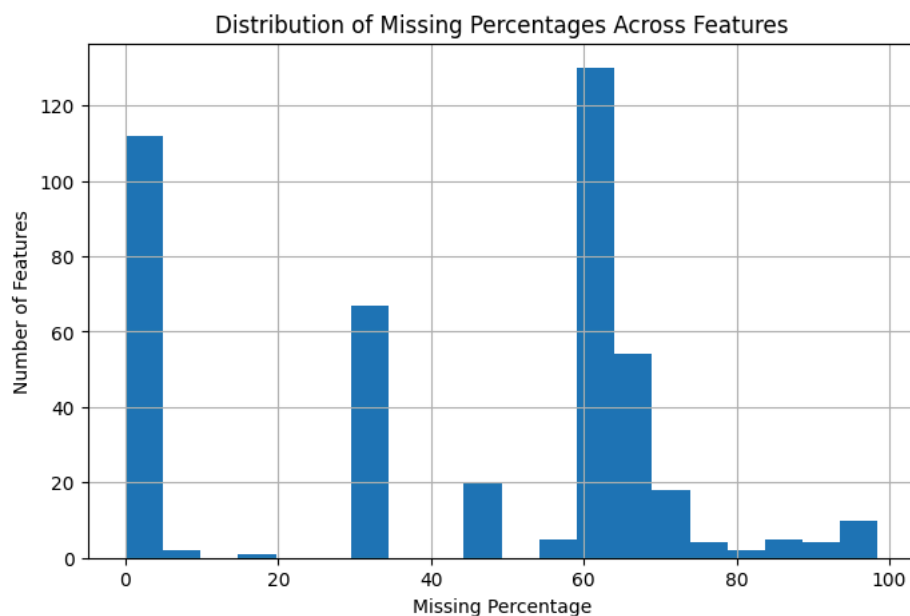
missing_pct.hist(
    bins=20
)

plt.title(
    "Distribution of Missing Percentages Across Features"
)

plt.xlabel(
    "Missing Percentage"
)

plt.ylabel(
    "Number of Features"
)

plt.show()
```



Observation

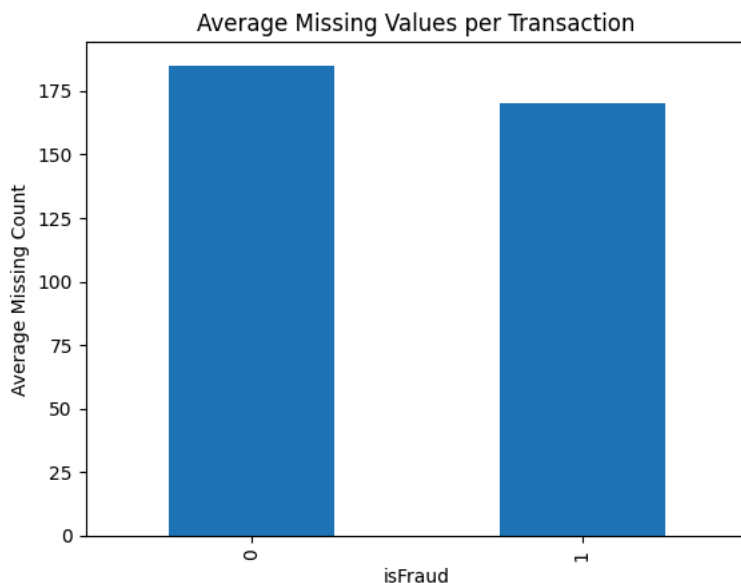
The dataset exhibits highly uneven missing-value patterns.

While many features contain little or no missing information, a substantial number of variables contain more than 60% missing values, and several identity-related features are almost entirely missing.

This suggests that missingness may itself contain useful information and should be investigated before feature removal or imputation.

```
df["missing_count"] = (\n    df.isnull()\n    .sum(axis=1)\n)\n\nfraud_missing = (\n    df.groupby("isFraud")\n    ["missing_count"]\n    .mean()\n)\n\nprint(fraud_missing)\n\nisFraud\n0    184.999528\n1    170.273331\nName: missing_count, dtype: float64
```

```
fraud_missing.plot(\n    kind="bar"\n)\n\nplt.title(\n    "Average Missing Values per Transaction"\n)\n\nplt.ylabel(\n    "Average Missing Count"\n)\n\nplt.show()
```



Finding

Fraudulent transactions contain fewer missing values on average than legitimate transactions.

This indicates that overall missing-value volume alone is not a strong fraud indicator.

However, specific variables may still exhibit fraud-related missingness patterns and will be investigated later during feature analysis.

This finding highlights the importance of analyzing missingness at the feature level rather than relying solely on aggregate missing counts.

Feature Reduction

High-dimensional datasets often contain redundant, constant, or highly correlated variables.

Reducing feature count can improve model efficiency, reduce overfitting risk, and simplify model interpretation while maintaining predictive performance.

```
print("Original Shape:")  
print(df.shape)
```

```
Original Shape:  
(100000, 435)
```

Constant Feature Analysis

Features with only a single unique value provide no predictive information and can be safely removed.

```
constant_cols = [  
    col  
    for col in df.columns  
    if df[col].nunique() <= 1  
]  
  
print(  
    "Constant Features:",  
    len(constant_cols)  
)
```

```
Constant Features: 2
```

Finding

2 constant features were identified.

Because these variables contain no variation across transactions, they provide no predictive value and were removed before model training.

Near-Constant Feature Analysis

Features that contain the same value for the vast majority of observations contribute little information and may increase model complexity.

These variables were evaluated and removed where appropriate.

```
constant_cols = []  
near_constant_cols = []  
  
for col in df.columns:  
  
    unique_count = df[col].nunique(dropna=False)  
  
    if unique_count == 1:  
        constant_cols.append(col)  
  
    else:  
        top_freq = (  
            df[col]  
            .value_counts(  
                normalize=True,  
                dropna=False  
            )  
            .iloc[0]  
        )  
  
        if top_freq > 0.99:
```

```

near_constant_cols.append(col)

print("Constant Features:", len(constant_cols))
print("Near Constant Features:", len(near_constant_cols))

```

```

Constant Features: 1
Near Constant Features: 15

```

Finding

1 constant feature and 15 near-constant features were identified.

Constant features provide no predictive information and were removed.

Near-constant features were flagged for further evaluation because rare categories may still contain useful fraud-related patterns.

Correlation Analysis

Many features may contain overlapping information.

To reduce redundancy and improve model efficiency, pairwise correlations were analyzed and highly correlated variables were identified.

```

num_cols = df.select_dtypes(include=np.number).columns

corr_matrix = df[num_cols].corr().abs()

upper = corr_matrix.where(
    np.triu(
        np.ones(corr_matrix.shape),
        k=1
    ).astype(bool)
)

high_corr_pairs = []

for col in upper.columns:
    for row in upper.index:

        if pd.notnull(upper.loc[row, col]) and upper.loc[row, col] > 0.95:

            high_corr_pairs.append(
                (
                    row,
                    col,
                    round(upper.loc[row, col], 3)
                )
            )

print("Highly Correlated Pairs:", len(high_corr_pairs))

```

```

Highly Correlated Pairs: 298

```

Finding

298 highly correlated feature pairs were identified.

This indicates that several variables contain overlapping information and may contribute unnecessary complexity to the model.

Feature reduction was therefore performed to eliminate redundancy while preserving predictive signal.

```

to_drop_corr = [
    column
    for column in upper.columns
    if any(upper[column] > 0.95)
]

print("Features To Remove:", len(to_drop_corr))

```

```

Features To Remove: 126

```

Feature Reduction Result

Correlation analysis identified 298 highly correlated feature pairs.

Using a correlation threshold of 0.95, 126 redundant features were removed.

This reduced dimensionality while retaining the majority of predictive information, resulting in a more efficient feature set for model training.

```
df_reduced = df.drop(
    columns=to_drop_corr
)

print("Original Shape:", df.shape)
print("Reduced Shape:", df_reduced.shape)
```

```
Original Shape: (100000, 435)
Reduced Shape: (100000, 309)
```

Final Feature Set

The feature reduction process removed 126 highly correlated variables and reduced the dataset from 435 to 309 features.

This represents a 29% reduction in dimensionality while preserving the majority of available predictive information.

The reduced feature set was used for all subsequent modeling experiments.

Data Preparation

Before training the fraud detection model, categorical variables were encoded and the dataset was divided into training and testing subsets.

A stratified train-test split was used to maintain the original fraud distribution across both datasets.

```
from sklearn.preprocessing import LabelEncoder

label_encoders = {}

for col in df_reduced.select_dtypes(include="object").columns:

    le = LabelEncoder()

    df_reduced[col] = (
        df_reduced[col]
        .astype(str)
    )

    df_reduced[col] = le.fit_transform(
        df_reduced[col]
    )

    label_encoders[col] = le
```

```
X = df_reduced.drop(
    columns=["isFraud"]
)

y = df_reduced["isFraud"]

print(X.shape)
print(y.shape)
```

```
(100000, 308)
(100000,)
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2
```

```

    stratify=y,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)

(80000, 308)
(20000, 308)

```

Result

The dataset was split using an 80:20 ratio.

Training Set: 80,000 transactions

Testing Set: 20,000 transactions

Stratification ensured that both datasets preserved the original fraud rate of approximately 2.56%.

✖ XGBoost Fraud Detection Model

XGBoost was selected because of its strong performance on structured tabular datasets and its ability to capture complex non-linear relationships.

The model was trained on the reduced feature set and evaluated using precision, recall, and ROC-AUC.

```

from xgboost import XGBClassifier

reduced_model = XGBClassifier(
    n_estimators=400,
    max_depth=8,
    learning_rate=0.05,
    colsample_bytree=0.7,
    eval_metric="logloss",
    random_state=42
)

reduced_model.fit(
    X_train,
    y_train
)

reduced_model

```

```

XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=0.7, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.05, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=8, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=400, n_jobs=None,
              num_parallel_tree=None, ...)

```

✖ Model Predictions

After training, the model generates a fraud probability score for each transaction.

A default classification threshold of 0.50 is initially used to classify transactions as fraud or non-fraud.

```

y_prob = reduced_model.predict_proba(
    X_test
)[: , 1]

pred = (
    y_prob >= 0.5

```



```
).astype(int)

print("Predictions Generated")
```

Predictions Generated

Model Evaluation

The fraud detection model was evaluated using:

- Precision
- Recall
- F1-Score
- ROC-AUC
- Confusion Matrix

These metrics provide a balanced assessment of fraud detection performance on an imbalanced dataset.

```
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_auc_score
)

print(
    classification_report(
        y_test,
        pred
    )
)

print(
    confusion_matrix(
        y_test,
        pred
    )
)

print(
    "ROC-AUC:",
    roc_auc_score(
        y_test,
        y_prob
    )
)
```

	precision	recall	f1-score	support
0	0.99	1.00	0.99	19488
1	0.95	0.49	0.65	512
accuracy			0.99	20000
macro avg	0.97	0.74	0.82	20000
weighted avg	0.99	0.99	0.98	20000
[[19476 12]				
[262 250]]				
ROC-AUC: 0.9432933788581435				

Model Performance

The XGBoost model achieved a ROC-AUC score of 0.943, demonstrating strong discrimination between fraudulent and legitimate transactions.

Key Results:

- Precision: 95%
- Recall: 49%
- Fraud Cases Detected: 250 / 512
- False Positives: 12
- Overall Accuracy: 98.6%

The model is highly precise, meaning that when a transaction is flagged as fraud, it is usually correct. However, the default threshold prioritizes precision over recall, causing a significant number of fraudulent transactions to remain undetected.

This motivates threshold optimization in the next stage.

Threshold Optimization

Fraud detection systems typically prioritize recall over precision because missing fraudulent transactions can be more costly than investigating additional legitimate transactions.

Instead of using the default threshold of 0.50, multiple thresholds were evaluated to identify a better balance between fraud detection and review workload.

```
from sklearn.metrics import precision_score, recall_score

thresholds = [0.5, 0.4, 0.3, 0.2]

results = []

for t in thresholds:

    pred = (y_prob >= t).astype(int)

    precision = precision_score(
        y_test,
        pred
    )

    recall = recall_score(
        y_test,
        pred
    )

    results.append(
        [t, precision, recall]
    )

threshold_df = pd.DataFrame(
    results,
    columns=[
        "Threshold",
        "Precision",
        "Recall"
    ]
)

threshold_df
```

	Threshold	Precision	Recall	
0	0.5	0.954198	0.488281	
1	0.4	0.931655	0.505859	
2	0.3	0.881250	0.550781	
3	0.2	0.813514	0.587891	

Next steps:

Generate code with threshold_df

New interactive sheet

Threshold Optimization Results

Lowering the classification threshold increased fraud detection recall at the expense of precision.

Threshold	Precision	Recall
0.50	95.4%	48.8%
0.40	93.2%	50.6%
0.30	88.1%	55.1%
0.20	81.4%	58.8%

Reducing the threshold from 0.50 to 0.20 increased fraud recall from 48.8% to 58.8%, enabling the system to detect 20% more fraudulent transactions while maintaining strong precision.

This highlights the trade-off between fraud detection effectiveness and investigation workload.

✓ Cost-Aware Fraud Detection

Traditional fraud detection models rank transactions solely by fraud probability.

However, in real-world environments, missing a high-value fraudulent transaction is often more costly than missing a low-value one.

To prioritize financial impact, a cost-aware ranking strategy was developed using both fraud probability and transaction amount.

```
test_results = X_test.copy()

test_results["Actual_Label"] = y_test.values
test_results["Fraud_Probability"] = y_prob
test_results["TransactionAmt"] = (
    df_reduced.loc[
        X_test.index,
        "TransactionAmt"
    ]
)
```

```
test_results["LossScore"] = (
    test_results["Fraud_Probability"]
    *
    test_results["TransactionAmt"]
)

test_results[
    [
        "Fraud_Probability",
        "TransactionAmt",
        "LossScore"
    ]
].head()
```

	Fraud_Probability	TransactionAmt	LossScore	
30380	0.001335	25.000	0.033370	
19535	0.002423	100.000	0.242284	
33142	0.005275	4.839	0.025526	
90781	0.002277	117.000	0.266433	
19818	0.004751	59.000	0.280338	

LossScore Interpretation

Fraud probability alone does not capture the financial impact of a fraudulent transaction.

To address this limitation, a LossScore was created by combining fraud probability with transaction value:

LossScore = Fraud Probability × Transaction Amount

This approach prioritizes transactions that are both likely to be fraudulent and financially significant.

As a result, investigation resources can be focused on transactions with the highest potential financial loss rather than simply the highest fraud probability.

✓ Review Budget Simulation

Fraud investigation teams typically operate under limited review capacity.

To evaluate the practical value of LossScore, a review-budget simulation was conducted.

The objective was to compare:

1. Ranking transactions by Fraud Probability

2. Ranking transactions by LossScore

under the same investigation budget.

✓ Budget Selection

The review budgets (100, 250, 500, and 1000 transactions) were chosen to represent progressively larger investigation capacities.

Given a test dataset of 20,000 transactions, these budgets correspond to reviewing approximately:

- 0.5% of transactions
- 1.25% of transactions
- 2.5% of transactions
- 5% of transactions

This allows evaluation of fraud recovery performance under different operational constraints.

```
budgets = [100, 250, 500, 1000]

results = []

for b in budgets:

    top_prob = test_results.sort_values(
        "Fraud_Probability",
        ascending=False
    ).head(b)

    top_loss = test_results.sort_values(
        "LossScore",
        ascending=False
    ).head(b)

    prob_value = top_prob[
        top_prob["Actual_Label"] == 1
    ]["TransactionAmt"].sum()

    loss_value = top_loss[
        top_loss["Actual_Label"] == 1
    ]["TransactionAmt"].sum()

    improvement = (
        (loss_value - prob_value)
        / prob_value
        * 100
    )

    results.append(
        [
            b,
            round(prob_value, 2),
            round(loss_value, 2),
            round(improvement, 2)
        ]
    )

budget_df = pd.DataFrame(
    results,
    columns=[
        "Review_Budget",
        "Probability_Value",
        "LossScore_Value",
        "Improvement_%"
    ]
)

budget_df
```

	Review_Budget	Probability_Value	LossScore_Value	Improvement_%	
0	100	14043.02	26672.91	89.94	
1	250	29474.82	35883.28	21.74	
2	500	43561.34	49384.02	13.37	
3	1000	49455.31	60490.80	22.31	

Next steps: [Generate code with budget_df](#)[New interactive sheet](#)

Business Impact Evaluation

The review-budget simulation demonstrates that LossScore consistently recovers more fraudulent transaction value than ranking transactions solely by fraud probability.

Key Results:

- Top 100 Reviews:
 - Fraud Probability Ranking: 14, 043 — *LossScoreRanking* :26,673
 - Improvement: +89.9%
- Top 250 Reviews:
 - Improvement: +21.7%
- Top 500 Reviews:
 - Improvement: +13.4%
- Top 1000 Reviews:
 - Improvement: +22.3%

The largest gains occur when investigation resources are most constrained. With a review budget of only 100 transactions, LossScore nearly doubles the amount of fraud value recovered.

These results demonstrate that incorporating financial impact into transaction prioritization can significantly improve fraud-loss recovery without increasing analyst workload.

Feature Importance Analysis

Understanding which variables influence fraud predictions is important for model transparency and business trust.

Feature importance analysis was performed to identify the variables that contributed most strongly to fraud detection decisions.

```
import pandas as pd

feature_importance = pd.DataFrame({
    "Feature": X_train.columns,
    "Importance": reduced_model.feature_importances_
})

feature_importance = (
    feature_importance
    .sort_values(
        "Importance",
        ascending=False
    )
)

feature_importance.head(20)
```

	Feature	Importance	
176	V201	0.101519	
162	V173	0.066674	
203	V242	0.056350	
171	V189	0.051729	
170	V188	0.044339	
150	V148	0.026149	
5	card3	0.022907	
209	V257	0.022423	
190	V223	0.020234	
256	V323	0.020188	
173	V197	0.015086	
252	V317	0.013505	
151	V157	0.013344	
174	V198	0.009005	
140	V134	0.008132	
17	C5	0.008050	
2	ProductCD	0.007889	
76	V45	0.007443	
283	id_17	0.007436	
160	V171	0.007000	

Next steps: [Generate code with feature_importance](#) [New interactive sheet](#)

Feature Importance Findings

The model relied heavily on engineered transaction features, particularly the V-series variables generated from underlying transactional relationships.

Top predictive features included:

- V201
- V173
- V242
- V189
- V188

In addition to engineered variables, traditional transaction attributes such as:

- card3
- ProductCD
- C5
- id_17

also contributed to fraud prediction.

This suggests that fraud behavior is influenced by a combination of transaction patterns, payment-card characteristics, and identity-related information.

```
import matplotlib.pyplot as plt

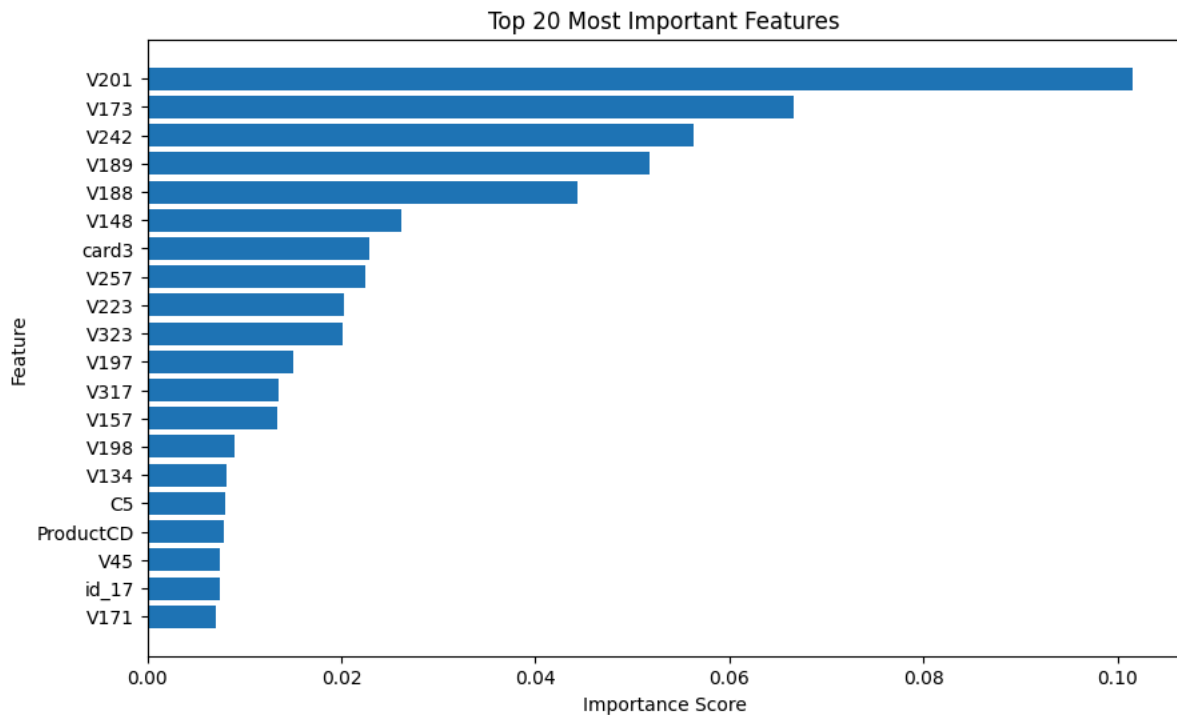
top20 = feature_importance.head(20)

plt.figure(figsize=(10,6))

plt.barh(
    top20["Feature"][:-1],
    top20["Importance"][:-1]
)
```

```
plt.title("Top 20 Most Important Features")
plt.xlabel("Importance Score")
plt.ylabel("Feature")

plt.show()
```



Key Finding

Engineered V-features were the strongest predictors of fraud, with V201 contributing the highest importance score.

Traditional transaction variables such as ProductCD, card3, and identity-related attributes also influenced fraud predictions.

This suggests that both behavioral patterns and transaction context play important roles in identifying fraudulent activity.

```
from sklearn.metrics import fbeta_score
import numpy as np

thresholds = np.arange(0.05, 0.95, 0.05)

best_threshold = 0
best_f2 = 0

for t in thresholds:
    pred = (y_prob >= t).astype(int)

    f2 = fbeta_score(
        y_test,
        pred,
        beta=2
    )

    if f2 > best_f2:
        best_f2 = f2
        best_threshold = t

print("Best Threshold:", best_threshold)
print("Best F2:", round(best_f2, 4))
```

```
Best Threshold: 0.1
Best F2: 0.6667
```

```
best_pred = (
    y_prob >= 0.1
)
```

```
astype(int)
```

```
print(classification_report(y_test, best_pred))
print(confusion_matrix(y_test, best_pred))
```

```

              precision    recall  f1-score   support

     0       0.99      0.99      0.99     19488
     1       0.63      0.68      0.65       512

 accuracy      0.98
 macro avg      0.81
 weighted avg    0.98

[[19287  201]
 [  166  346]]
```

SHAP Explainability

While feature importance identifies which variables contribute most to the model, it does not explain how individual features influence predictions.

SHAP (SHapley Additive exPlanations) was used to provide model transparency and identify the features driving fraud predictions.

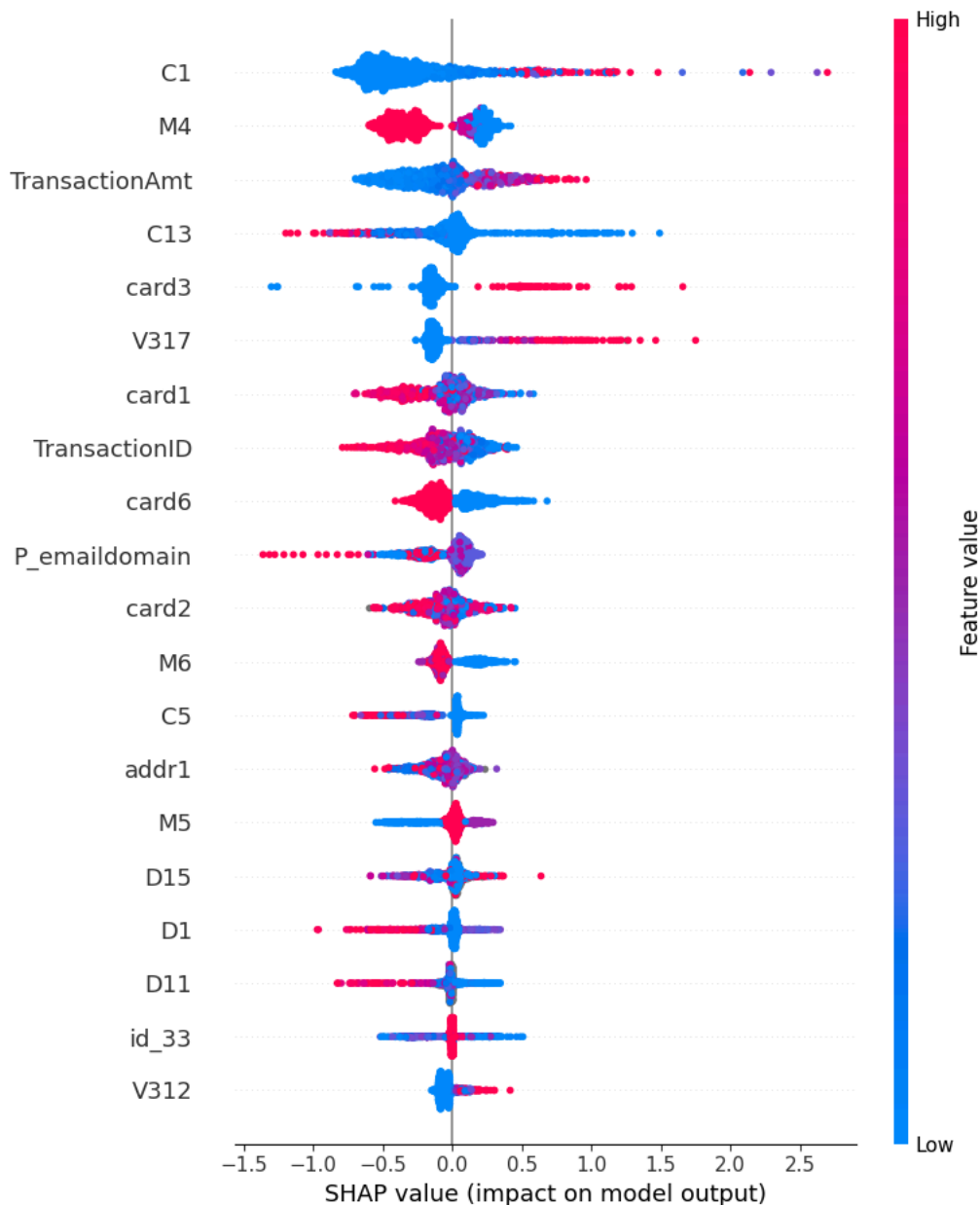
```

explainer = shap.TreeExplainer(
    reduced_model
)

sample_data = X_test.sample(
    1000,
    random_state=42
)

shap_values = explainer.shap_values(
    sample_data
)

shap.summary_plot(
    shap_values,
    sample_data
)
```

SHAP Business Interpretation

The SHAP summary plot shows that C1, M4, and TransactionAmt are the most influential features in fraud detection. Features such as C1, card3, and V317 have higher values that generally increase the fraud prediction score, while lower values of C13 tend to increase fraud likelihood. The spread of SHAP values indicates that these features have a strong impact on the model's decisions. Overall, the model relies on a combination of transaction behavior, card information, and engineered features to distinguish fraudulent transactions from legitimate ones.

```
# Analyze False Negatives
fn_idx = y_test[(y_test == 1) & (best_pred == 0)].index
false_negatives = X_test.loc[fn_idx]

false_negatives["TransactionAmt"].describe()
```

	TransactionAmt
count	166.000000
mean	177.605259
std	318.477480
min	0.467000
25%	49.250000
50%	88.250000
75%	171.000000
max	3081.970000

dtype: float64

Observation:

The average transaction amount is 177.61, while the median is 88.25. Since the mean is much higher than the median and the maximum value (3081.97) is significantly larger than most transactions, the data is positively (right) skewed. This suggests the presence of high-value transactions/outliers that increase the average transaction amount.

```
from sklearn.metrics import fbeta_score
import numpy as np

thresholds = np.arange(
    0.05,
    0.95,
    0.05
)

best_threshold = 0
best_f2 = 0

for t in thresholds:

    pred = (
        y_prob >= t
    ).astype(int)

    f2 = fbeta_score(
        y_test,
        pred,
        beta=2
    )

    if f2 > best_f2:
        best_f2 = f2
        best_threshold = t

print(
    "Best Threshold:",
    best_threshold
)

print(
    "Best F2:",
    round(best_f2,4)
)
```

Best Threshold: 0.1
Best F2: 0.6667

```
best_pred = (
    y_prob >= best_threshold
).astype(int)

print(
    classification_report(
        y_test,
        best_pred
    )
)
```

```

)

print(
    confusion_matrix(
        y_test,
        best_pred
    )
)

```

	precision	recall	f1-score	support
0	0.99	0.99	0.99	19488
1	0.63	0.68	0.65	512
accuracy			0.98	20000
macro avg	0.81	0.83	0.82	20000
weighted avg	0.98	0.98	0.98	20000


```

[[19287  201]
 [   166  346]]

```

F2-Based Threshold Optimization

The default threshold of 0.50 prioritized precision and achieved a recall of 48.8%.

To improve fraud detection coverage, thresholds were optimized using the F2 score, which places greater emphasis on recall.

Results

- Best Threshold: 0.10
- Best F2 Score: 0.667
- Precision: 63.0%